

Local-First Teams of Coding Agents:

A Field Study of 344 Merged Tasks in Nine Days with a Supervised PM / Tech-Lead / Coder Architecture

Ali Mohammad
Independent researcher
<email>

October 2026

Abstract

Coding agents now resolve isolated issues well, but a product is not a set of isolated issues: it needs someone to decide what is wrong, someone to write the fix, someone to check it, and a way for all of them to share one repository without colliding. We present **ATT** (Agent Team Template), an open, *local-first* operating model for a team of Claude Code agents running on a single developer machine: a product manager (PM) agent that runs the product and files provable backlog rows, several coding agents that claim rows and open pull requests, several tech-lead agents that review and merge them behind a guarded merge, and an agent manager that the human owner talks to. Coordination uses only artefacts a developer already has (a Markdown backlog, git commits, GitHub labels, file locks), and every agent runs as a chain of short, fresh sessions under a supervisor that survives usage limits. We report a field study of one local deployment on a private, actively developed project. Over nine days, about 430 backlog rows were filed and 344 claimed rows were merged, at a median of **1.3 hours** from claim to merge (IQR 0.6–3.2 h; 91% within 8 h), with two reverts on the integration branch. Once the supervisor was introduced, median claim-to-merge latency fell from 10.7 h ($n=30$) to 1.2 h ($n=314$). This comparison is observational and confounded. We describe the design, the measurement method, the 83 failure-driven “lessons” the agents accumulated, and the threats to validity. We also propose a controlled benchmark protocol for future work. The template is released as open source.

1 Introduction

Benchmarks such as SWE-bench [Jimenez et al., 2024] made autonomous issue resolution measurable, and agent scaffolds such as SWE-agent [Yang et al., 2024], OpenHands [Wang et al., 2024] and Agentless [Xia et al., 2024] have pushed resolution rates up quickly. These settings share an assumption: a task arrives fully specified, one agent works on it alone, and success is a hidden test. Day-to-day product work differs on each count. Someone must *find* what is wrong with the product and write it down so that it can be checked. Several fixes are in flight at once, against one moving branch. Someone other than the author must check each fix before it lands. And everything runs on the developer’s own machine and CI, whose memory and time are finite.

Multi-agent frameworks such as MetaGPT [Hong et al., 2023], ChatDev [Qian et al., 2023] and AutoGen [Wu et al., 2023] model software roles inside one conversation or one process. ATT takes a different position: **the roles are separate, long-lived processes, and they coordinate only**

through the repository. A backlog row, a claim commit, a PR label and a lock file are the whole protocol. This has three consequences that we found decisive in practice:

1. *Context stays small.* No agent holds the team’s state in its context. Each session does one unit of work and exits, which avoids the known degradation of long contexts [Liu et al., 2023].
2. *It is local-first.* The team runs on one owner machine, under `systemd` user units, against the owner’s own CI runners. No hosted orchestration service is involved, and the owner can inspect or stop any agent with ordinary tools.
3. *Failures become rules.* Each role keeps a numbered `lessons.md`. A failure that cost a run becomes a written rule that later sessions read, which is a persistent, human-auditable analogue of verbal self-reflection [Shinn et al., 2023].

Contributions. (i) A role and protocol design for a team of coding agents sharing one repository and one machine (§2). (ii) An open-source implementation for Claude Code [Anthropic, 2025]: skills, shared docs and a supervisor [Mohammad, 2026]. (iii) A field study of one deployment, measured from git history alone (§3–4). (iv) A catalogue of the failure modes that the agents’ lessons record (§5), and a benchmark protocol for a controlled evaluation (§7).

2 Design

2.1 Roles

Role	Does	Never
PM	runs the product end to end on real inputs; judges the output; files numbered rows with a level and a <i>done when</i> ; in <i>goal mode</i> , researches a feature into a brief and rows	reviews or merges PRs
Coding agent	claims a ready row at its level; writes a failing test; fixes the row on agent/NN-slug ; opens a PR in a fixed template; at most two rows per session	merges; writes rows; runs heavy work locally
Tech lead	reviews PRs against the row’s <i>done when</i> , using the diff, the PR’s tests and CI; starts a guarded merge; builds merge trains; keeps CI green	runs the product; numbers rows
Agent manager	the owner’s own session: starts, stops and reports on the team; relays decisions; fixes the machinery	the agents’ own work

2.2 The backlog protocol

A single Markdown file, `docs/BACKLOG.md`, is the only list of open work. Each part of it has exactly one writer. The PM writes rows, their *level* (L1: one or two files with the fix named; L2: logic across a few files following a pattern; L3: a new instrument or work that must be proven on real fixtures), and their *done when*, which must name a real artefact and a number. A coding agent’s only edit is the **claim**: a commit changing one state cell from **ready** to **claimed agent/NN-slug**, pushed directly to the integration branch. Two agents claiming the same row is a git push race that one of them loses, so mutual exclusion comes from git itself. Leads own a row’s state while its PR is open, and the *Merged* table. The *Inbox* is a one-way channel from the leads to the PM.

2.3 Sessions and the supervisor

Every agent is a chain of short sessions. A session reads a per-agent *handoff* file, does one unit of work (a coder: finish open PRs, then at most two claims; a lead: review until nothing is left to take; the PM: one live run or one backlog pass), rewrites the handoff, and writes one status line, `DONE`, `IDLE` or `BLOCKED`. A Bash supervisor (about 450 lines) runs each agent under a `systemd` user unit. It creates and signs the agent’s git worktree, gates new sessions on free memory and load, waits out account usage limits, resumes the *same* session after the reset, and stops the agent and notifies the owner on `BLOCKED`. The PM and the leads run as interactive background sessions with Remote Control enabled, so the owner can join any of them from a phone; the supervisor keeps them open for a grace period after their status line.

2.4 Concurrency on one machine

Each agent works in its own git worktree, detached on the integration branch, and signs its commits with a per-agent e-mail suffix, so authorship is attributable even though every agent pushes as the same GitHub user. Three file locks serialise what must be serial: pushes to the integration branch, heavy work (exports, renders, heavy tests), and the single application server used for live runs. GitHub labels (`review:<lead>`, `coder:<agent>`) act as visible, expiring leases on PRs. Memory is treated as the PM’s: coding agents run only a capped quick test on their own new test file (512 MB, 2 min), and CI is their real test run.

2.5 Guarded merges and trains

A lead never merges by hand. A detached job waits for CI, merges only if CI is green *on the exact head the lead reviewed* (`-match-head-commit`), requires at least one real `SUCCESS` (a draft’s all-`SKIPPED` rollup must not read as a pass), deletes the branch only after GitHub reports `MERGED`, and then writes the *Merged* row under the push lock. When CI time became the bottleneck, the leads switched to *merge trains*: approved PRs stay drafts, one lead merges several of them into `train/NN`, and a single CI run covers the whole set.

3 Deployment and method

Setting. One owner deployed the template locally, on their own machine, for a private software project under active development. The machine had 16 cores and 14 GB of RAM and also ran the owner’s own applications. CI ran on self-hosted runners: the owner’s PC and several laptops, later joined by a rented server. Team size varied with the owner’s instructions. Over the window we observe 14 signed agent identities: up to five supervised coding agents, an IDE-hosted coding agent, three tech leads, a run-mode PM, a goal-mode PM and a QA PM, plus unsigned cloud sessions. Coding agents used Opus-class models by default and Sonnet-class models for L1 rows.

Data. All numbers come from the integration branch’s git history between 2026-09-25 and 2026-10-08 (3,380 commits). Claim commits (`Claim #NN (Ln) ...`), guarded-merge commits (`Merge #NN (PR #N)`) and train commits (`Merge train/NN (PR #N): #a #b ...`) are written by the agents and scripts in fixed formats. Authors are attributed by their signed e-mail.

Metrics. *Claim-to-merge latency* is the time from a row’s first claim commit to the commit that records it in *Merged*. It covers coding, CI, review and merge, and is measured only for rows that have both commits. *Throughput* is the number of rows recorded as merged per calendar day.

4 Results

Rows numbered by the PM in the window (#89–#520)	≈432
Distinct rows claimed by coding agents	365
of which merged	344 (94%)
Rows recorded as merged (all paths)	373
GitHub PR merges into the integration branch	344
merge trains / rows merged via trains	21 / 177
Reverts on the integration branch	2
Lead commits mentioning “changes requested”	186
Lessons recorded (tech lead / PM)	56 / 27

Table 1: Volume over the study window.

Latency. The median claim-to-merge latency over 344 rows was **1.27 h** (IQR 0.63–3.19 h, p90 6.1 h, mean 4.2 h). 66% of rows merged within 2 h and 91% within 8 h; 19 rows took more than a day (Figure 1). Latency barely depends on level: medians were 1.21 h for L1 ($n=139$), 1.25 h for L2 ($n=175$) and 1.41 h for L3 ($n=30$). Once a row is claimed, the time is dominated by CI and review queues rather than by coding difficulty. This matches the team’s own diagnosis (§5).

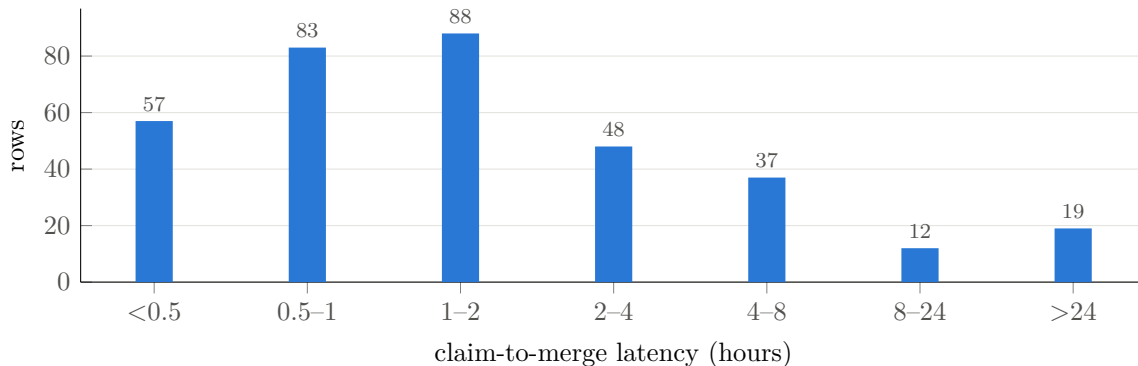


Figure 1: Distribution of claim-to-merge latency for the 344 rows that were both claimed and merged.

Throughput. Merged rows per day rose from 6–16 before the supervisor (introduced on 2026-10-04) to a peak of 147 on 2026-10-05, the day the leads began merging in trains (Figure 2). Of the 373 merged rows, 33 landed before the supervisor and 340 after it.

Before and after the supervisor. For rows claimed before 2026-10-04, median claim-to-merge latency was 10.7 h ($n=30$); for rows claimed after it, 1.2 h ($n=314$). Several things changed in the same days: more agents, a second and third lead, staged CI that tests only what a PR reaches, the

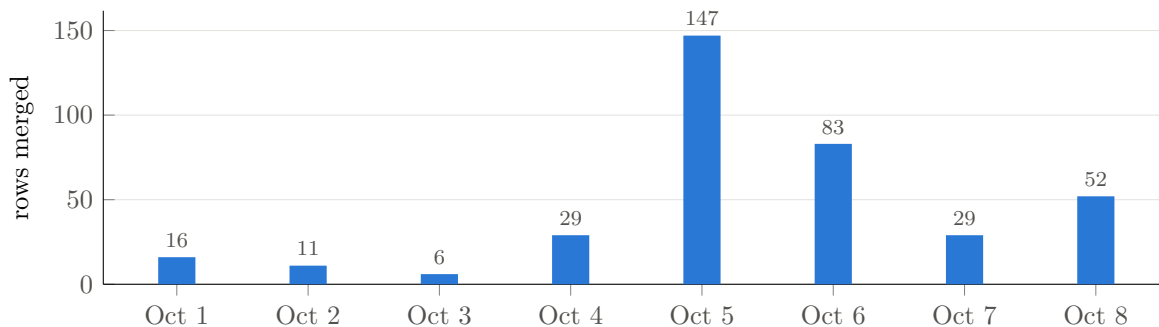


Figure 2: Backlog rows recorded as merged per day. The supervisor was introduced on Oct 4; merge trains on Oct 5. Oct 8 is a partial day.

quick test, and trains. We therefore read this as the *system* getting faster, not as an effect of the supervisor alone.

Who does the work. By signed commits, leads authored 1,110, supervised coders 726, the IDE-hosted coder 445, the PM agents 276 and cloud sessions 169. The remaining 654 are the owner’s, GitHub’s merge commits, or unsigned. Leads write the most commits because every review decision, row state change and *Merged* entry is a commit. The protocol records decisions as data rather than as chat.

5 What went wrong: the lessons

The 83 lessons are the most transferable output of the deployment. Each is dated and names the incident that taught it. Grouped by cause:

- **Shared state.** One process switching branches in a shared checkout put another agent’s commits on the wrong branch, which led to per-agent worktrees. Unsigned commits led a lead to misread other sessions’ merges as its own, which led to signed worktrees. Two leads reviewed the same PR, which led to label leases.
- **CI semantics.** A draft’s all-SKIPPED rollup read as “pass”. A cancelled run on the same head read as “fail”. A re-triggered run kept the old run’s cancelled jobs. A PR run kept the workflow file of its old merge commit. A CI job that never installed a toolchain skipped every test that needed it and went green. Each became a guard in the merge script or a rule in the lead’s skill.
- **Resources.** Three coders’ test suites, an orphaned application server and a supervisor test running together froze the 14GB machine. That led to “the memory is the PM’s”, capped quick tests, named memory scopes, and a self-hosted runner that leaves the queue while a live run is up.
- **The evaluation gap.** Fixes that passed their unit tests did nothing on the real path (a function only the UI called), tuned themselves to the one reference input, or were scored as fixed by the product’s own metric while failing on direct inspection. That led to “check the path, not the function”, “a fix’s own score is not an output test”, and owner notes that close only when a live run shows them fixed.

6 Limitations and threats to validity

This is a *field study of one deployment*, not a controlled experiment. (1) There is a single project, a single owner and nine days. (2) There is no control arm: we do not compare against one agent, a human team, or ATT without any one component. (3) The owner intervened throughout, setting priorities, freezing the backlog and resizing the team. (4) Latency is measured from commit timestamps; a row may have been worked on before its claim commit, and rows merged outside the guarded path lack a *Merged* commit. (5) Merged means CI-green and lead-approved. It does not mean correct: rows wait in *Merged* until a live run confirms them, and we do not report the confirmation rate here. (6) Commit counts are inflated by design, because every decision is a commit. They measure activity, not productivity.

7 Toward a controlled benchmark

We propose evaluating ATT, and team architectures generally, on *backlog-level* rather than issue-level tasks. Start each run from a frozen repository snapshot with a seeded backlog of N rows of mixed levels, some of which conflict in the files they touch, and a fixed machine and CI budget. Run ATT, a single agent working the same backlog sequentially, and ATT with one component removed (supervisor, leases, guarded merge, lessons). Measure rows merged per wall-clock hour, regressions introduced on the integration branch (by a held-out test suite), claim collisions, CI minutes per merged row, and tokens per merged row. SWE-bench-style hidden tests can serve as the held-out suite for each row.

8 Conclusion

A team of coding agents can run on one developer’s machine and keep up with a fast-moving product, provided the coordination is boringly explicit: one backlog with one writer per part, claims as commits, leases as labels, merges guarded on the reviewed head, short sessions with state on disk, and written lessons. In one deployment this sustained a median of 1.3h from claim to merge across 344 tasks. ATT packages that setup as a template, with every project-specific value in one configuration file.

Availability. <https://github.com/Alloooshe/ATT>

Acknowledgements. The agents described here, and parts of this manuscript’s drafting, used Claude models through Claude Code. All numbers were computed from the repository’s git history with the script `paper/analyze_git_history.py` in the ATT repository.

References

- Anthropic. Claude code. <https://claude.com/claude-code>, 2025.
- Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- Ali Mohammad. ATT: Agent team template. <https://github.com/Allooooshe/ATT>, 2026.
- Chen Qian et al. ChatDev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.